SET 6

PART A — Short questions and answers

Q1. Describe how the M:N relationship between LEARNER and COURSE is mapped to the relational schema.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare the DELETE and TRUNCATE commands.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain why storing several Course_IDs of a learner in one column violates 1NF.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain the use of the IN operator with a subquery.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Interpret the query: SELECT CourseID, AVG(ProgressPercent) FROM ENROLLMENT GROUP BY CourseID;

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Summarize the structure of a PL/SQL block (DECLARE, BEGIN, EXCEPTION, END).

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Compare the use of joins in SQL with embedded documents in MongoDB.

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q8. Describe how an array field such as Skills is stored in a document.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.courses.find({CourseLevel: "Beginner"}).sort({Price: 1})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (l:Learner)- [:ENROLLED_IN]->(c:Course) RETURN l.name, c.title;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An online learning company maintains a Online Learning Platform to store learner details, course details,

1(a) ER design:

• **LEARNER**(LearnerID, LearnerName, Email, Country); primary key: **LearnerID**.

• **COURSE**(CourseID, CourseTitle, CourseLevel, InstructorName, Price); primary key: **CourseID**.

• **ENROLLMENT**(EnrollmentID, LearnerID, CourseID, EnrollmentDate, ProgressPercent); primary key: **EnrollmentID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE LEARNER (
  LearnerID NUMBER PRIMARY KEY,
  LearnerName VARCHAR2(100),
  Email VARCHAR2(100),
  Country VARCHAR2(100)
);
CREATE TABLE COURSE (
  CourseID NUMBER PRIMARY KEY,
  CourseTitle VARCHAR2(100),
```

```
  CourseLevel VARCHAR2(100),
  InstructorName VARCHAR2(100),
  Price NUMBER
);
CREATE TABLE ENROLLMENT (
  EnrollmentID NUMBER PRIMARY KEY,
  LearnerID NUMBER,
  CourseID NUMBER,
  EnrollmentDate DATE,
  ProgressPercent NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM ENROLLMENT x
JOIN LEARNER a ON x.LearnerID = a.LearnerID
JOIN COURSE b ON x.CourseID = b.CourseID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```